DeployGuard

Final Year Project II — Implementation

Deterministic source-to-cloud deployment control plane

Implementation review • industrial visit • live deployment demonstration

Hasan Sajjad • **Faria Fatima** • **Tania Khawar**

Department of Computer Science • Air University Islamabad

Implementation baseline: test2 • reviewed 13 September 2026

What DeployGuard is

A self-service product that turns a supported repository into a verified cloud release.

Developer experience

Connect GitHub
Select branch and service
Configure environment
Deploy from one product surface

Control-plane responsibility

Freeze source identity
Resolve build/runtime contracts
Enforce policy and cost gates
Coordinate cloud lifecycle

Operational result

Immutable image
AWS runtime
Verified stable release
Rollback, destroy and evidence

DeployGuard’s contribution is the lifecycle and deterministic control plane—not the cloud services it orchestrates.

Problem being solved

Cloud deployment fails at the boundaries between tools, state and application assumptions.

Repository uncertainty

Which service? Which workspace? Which dependencies? Which application port?

Configuration drift

Source, build inputs, runtime variables and cloud state can diverge over time.

Operational uncertainty

A workflow can finish while the application is unhealthy, unreachable or misconfigured.

Recovery complexity

Redeploy, rollback and destroy need exact historical identity and ownership boundaries.

Security & cost

Deployment speed without security, identity and cost controls creates new failure modes.

The FYP-II implementation makes these boundaries explicit contracts instead of implicit assumptions.

Research foundation

Consensus literature was used to ground the implementation choices, while FYP-II focuses on the system we built.

Platform engineering

Self-service platforms reduce infrastructure cognitive load when treated as products with clear golden paths.
Anjum (2026); Pitkar (2026)

Infrastructure as Code

Declarative infrastructure improves repeatability and reduces configuration drift compared with manual provisioning.
Kasarla (2025)

DevSecOps

Embedding vulnerability scanning in delivery pipelines strengthens security before deployment.
Marandi et al. (2023); Kim et al. (2025)

Code-to-cloud automation

Research supports integrated source analysis, infrastructure generation and cloud deployment as an automation direction.
Odunsi et al. (2025)

AI + infrastructure operations

Recent reviews place AI across IaC generation, validation and operational analysis while highlighting the need for reliable evidence.
Pahl et al. (2026)

DeployGuard applies these principles through deterministic contracts, evidence-driven verification and explicit lifecycle ownership.

FYP-II implementation outcomes

The implementation spans the full deploy → operate → recover → destroy lifecycle.

Repository intelligence

exact SHA • service discovery • workspace boundaries

Build specification

canonical target • dependencies • package identity • fingerprint

Cloud lifecycle

GitHub Actions • OIDC • ECR • Terraform • ECS/Fargate

Runtime safety

ports • health • managed DB • release generations

Security

Trivy • secrets • RBAC • audit • capability verification

Operations

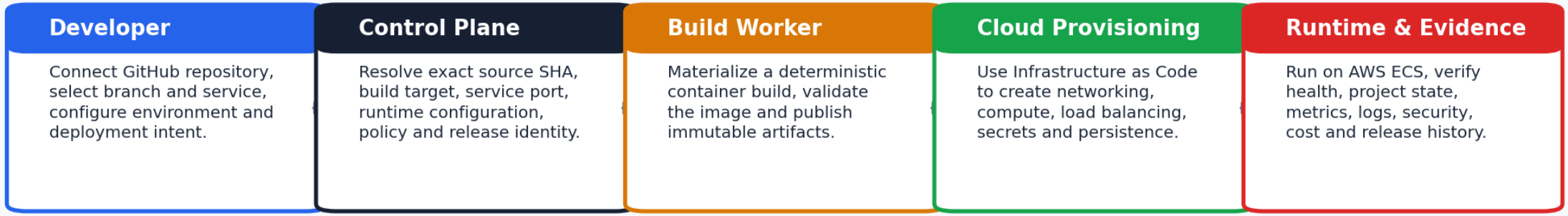
CloudWatch • monitoring • AI troubleshooting • FinOps • billing

Implementation architecture

A layered control plane converts repository intent into a verified runtime.

DeployGuard implementation architecture

High-level source-to-cloud flow used in the FYP-II implementation.



DeployGuard keeps source identity, deployment intent, runtime configuration and release evidence correlated across the lifecycle.

Product abstraction: Repository → canonical deployment specification → build worker → immutable image → Terraform/AWS → verified release.

Control-plane ownership

Clear ownership prevents Terraform, runtime orchestration and product state from fighting each other.

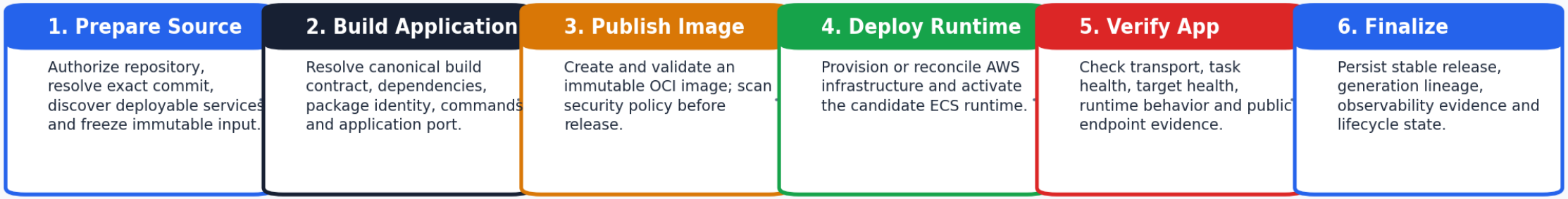
Layer	DeployGuard ownership	Execution dependency
Source & admission	exact commit, project/service identity, configuration readiness	GitHub repository
Build	canonical build target, port, runtime config, immutable identity	container build subsystem
Infrastructure	intent, capability checks, lifecycle admission	Terraform + AWS APIs
Runtime	success criteria, release promotion, failure classification	Amazon ECS / ALB
Operations	state, evidence, cost, diagnostics, rollback/destroy decisions	CloudWatch / ECR / Secrets / EFS

One authoritative owner per decision is a core implementation invariant.

Six-stage deployment pipeline

The user sees a stable lifecycle while internal work remains deeply automated.

Deployment lifecycle



DeployGuard keeps source identity, deployment intent, runtime configuration and release evidence correlated across the lifecycle.

Prepare Source → Build Application → Publish Image → Deploy Runtime → Verify Application → Finalize Release

Canonical build target

DeployGuard freezes “what should be built” before cloud mutation.

- Exact source SHA
- Service directory
- Workspace / build / install roots
- Package identity
- Deployment contract
- Execution metadata
- Dependency paths
- Evidence + SHA-256 fingerprint

Supported build contracts

JS_STANDALONE
JS_WORKSPACE_MEMBER
PYTHON_STANDALONE

Supported package managers:
npm • pnpm • yarn • bun

The immutable specification prevents different stages from re-interpreting the same repository differently.

Repository analysis and service selection

Fail closed when the service boundary is ambiguous or unsupported.

Repository
exact checkout

Service boundary
selected directory

Workspace evidence
manifests + lockfiles

Dependencies
local path analysis

Canonical target
resolved or blocked

Why this matters

Monorepos can contain multiple packages, sibling dependencies and conflicting package-manager evidence. DeployGuard creates one explicit boundary before build execution.

Admission behavior

Resolved → continue
Ambiguous / invalid / unsupported → stop with deterministic failure evidence

Automatic application-port authority

Port resolution follows evidence priority rather than a single framework guess.

Priority	Evidence	Examples
1	Explicit configuration	server.port • framework config
2	Source binding	listen(...) • app.run(...)
3	Start command	--port / -p
4	Environment	PORT / SERVER_PORT
5	Framework default	Next/Nest 3000 • Django/FastAPI 8000 • Flask 5000

Conflicting evidence is rejected instead of silently choosing an arbitrary port.

Container build subsystem

The platform owns the build contract; the worker materializes it into an OCI image.

Input
source SHA + canonical target

Resolve
install/build/start behavior

Build
configuration-driven worker

Validate
runtime image checks

Publish
immutable ECR artifact

Build-time boundary

Only declared build capabilities and dependencies affect build materialization.

Runtime boundary

PORT, HOST, runtime environment and secret references remain separate from build identity.

GitHub Actions and AWS identity

Short-lived OIDC federation replaces repository-specific long-lived AWS keys.



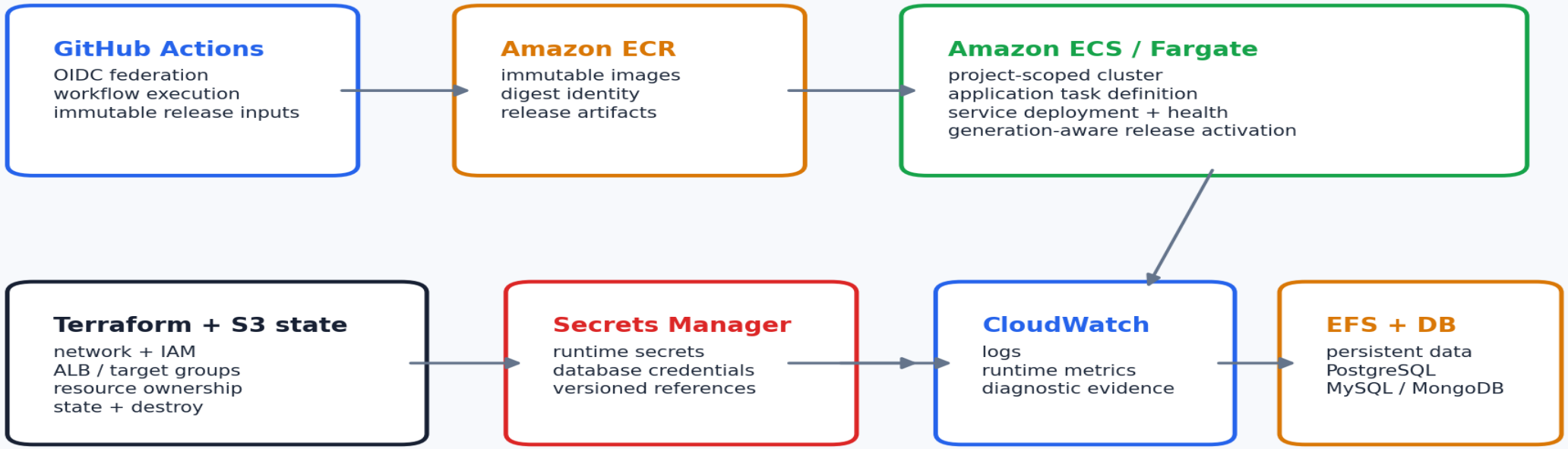
Before dispatch, DeployGuard validates trust and lifecycle-specific AWS capability requirements.

AWS runtime architecture

Terraform provisions infrastructure; DeployGuard verifies and promotes runtime success.

AWS runtime topology

Infrastructure is provisioned declaratively; DeployGuard remains the lifecycle and verification authority.



ALB exposes HTTP traffic; security groups isolate app and database paths; Cloud Map provides private database discovery.

Infrastructure ownership and state

Declarative infrastructure is a source of truth, but runtime release truth remains explicit.

Terraform owns

ECS resource existence
ALB / target groups
Security groups
IAM execution roles
EFS + database infra
Secrets infrastructure
Remote state + final destroy

DeployGuard owns

deployment admission
runtime readiness
active verified release
failure ownership
manual rollback target
cloud cleanup verification
state reconciliation

ECS owns

task scheduling
service rollout primitives
health/stability signals
active task execution
release update mechanics

Later infrastructure reconciliation must not unintentionally revert a verified active application release.

Managed database implementation

Database support is part of the deployment contract, not a manual post-deploy step.

Engine	Port	Persistence	Readiness
PostgreSQL 16	5432	EFS	pg_isready + runtime health
MySQL 8	3306	EFS	mysqladmin + credential/grant reconciliation
MongoDB 8	27017	EFS	authenticated ping

Secret injection

Database passwords and URLs are generated/stored through AWS Secrets Manager and referenced by the attached application service.

Network boundary

Database security-group ingress is restricted to the attached application service; private service discovery resolves the DB endpoint.

Security architecture

Security is embedded across identity, artifact, configuration and runtime stages.

Defense-in-depth deployment controls



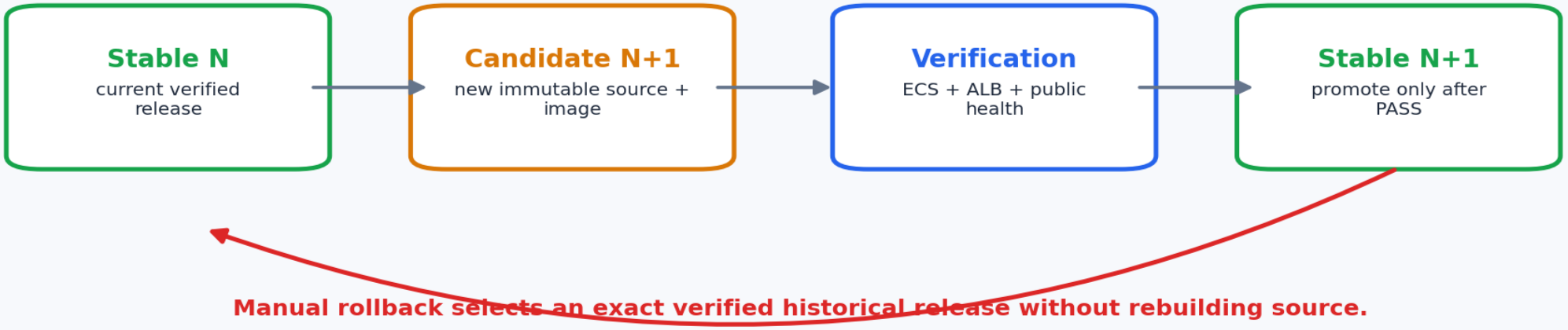
DeployGuard keeps source identity, deployment intent, runtime configuration and release evidence correlated across the lifecycle.

Vulnerability scanning can operate in visibility mode or enforcement mode before a release advances.

Release generations

A candidate is never “stable” merely because infrastructure exists.

Release generations and rollback



Database constraints enforce one live generation and one deploying candidate per project/environment.

Redeploy path

Code-only release changes avoid unnecessary full infrastructure reprovisioning.

Source change
new exact SHA

Build + scan
new immutable image

Task revision
register immutable runtime

ECS update
roll service

Verify + promote
stable only on PASS

Release-only updates preserve infrastructure ownership while advancing the active runtime revision.

Manual rollback

Rollback reuses a proven historical release instead of rebuilding old source.

Select
verified historical release

Validate
artifact + config compatibility

Activate
ECS previous revision

Verify
runtime + public health

Promote
move stable pointer

Key invariant

Rollback is an immutable release selection operation—not a source rebuild.

Destroy and cloud cleanup

Destruction is a verified lifecycle operation, not a blind delete call.

Confirm
explicit destructive admission

Destroy
Terraform resource deletion

Verify
AWS inventory reconciliation

Cleanup
non-IaC residue handling

Finalize
control-plane state closed

Project inventory, TTL cleanup and admin cleanup surfaces help detect orphaned resources after interrupted operations.

Observability and failure diagnostics

The product stores structured evidence so a failure can be classified, explained and recovered.

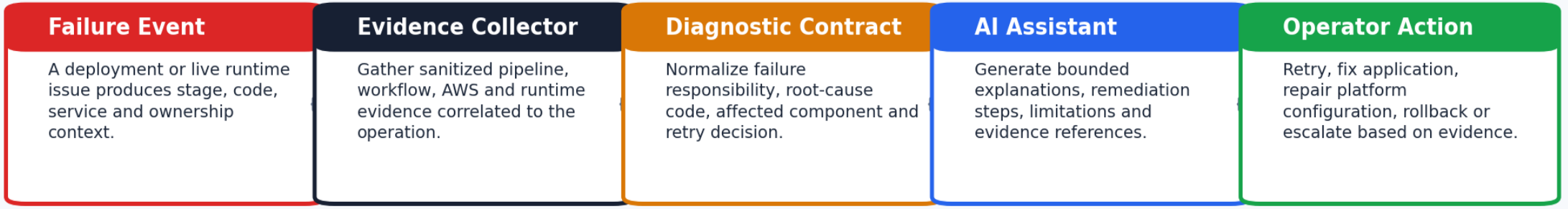
Evidence surface	Implementation
Pipeline	stage timing • status • terminal code • service identity
Workflow	run/job evidence • duration • dispatch state
AWS runtime	ECS events • stopped tasks • ALB target health • CloudWatch
Failure contract	owner • provider • root-cause code • retry decision
Monitoring	CPU/memory • latency • Prometheus/Grafana projections

Unknown evidence remains UNVERIFIED rather than being forced into a misleading root-cause category.

AI-assisted troubleshooting

AI is advisory; deterministic evidence remains authoritative.

Evidence-driven troubleshooting



DeployGuard keeps source identity, deployment intent, runtime configuration and release evidence correlated across the lifecycle.

Provider failure falls back to evidence-only diagnostics, preserving useful troubleshooting without inventing facts.

FinOps and billing

Cost is treated as a deployment policy dimension alongside security and technical readiness.

Cost evidence

Plan-derived estimate
resource breakdown
monthly total
change vs previous estimate
persisted cost history

Policy

warning threshold
subscription tier limit
approval-required state
block/resume pipeline
audit trail

Commercial layer

plans
subscriptions
checkout
invoices + payments
usage counters + entitlements

A technically valid deployment can still be paused when its predicted cost violates policy.

Verification and certification strategy

FYP-II evidence is built from executable contracts, not only screenshots.

Repository contracts

service discovery • build target • port • env isolation

Lifecycle

deploy • redeploy • rollback • destroy • state reconciliation

Security

OIDC • capability model • Trivy • secret boundaries

Runtime

ECS stability • health • multi-service • DB prerequisites

UI

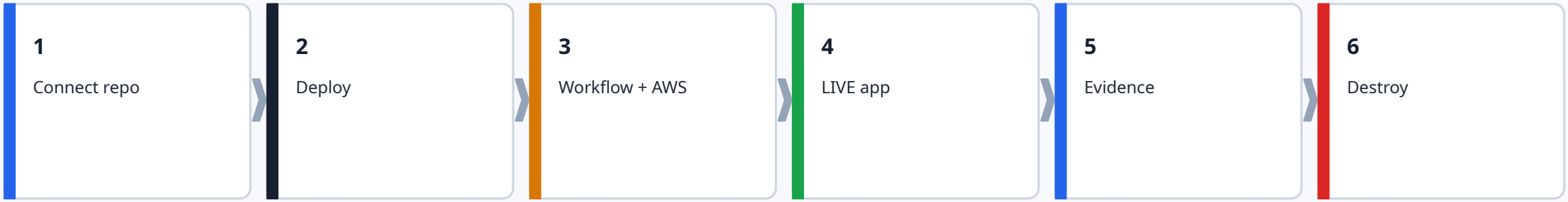
deployment journey • recovery • monitoring • destructive operations

Full stack

backend build • frontend build • integration • browser workflow

FYP-II live demonstration

Show the implementation as one coherent lifecycle.



- Keep one known-safe supported repository ready.
- Show exact commit / deployment identity before cloud execution.
- Open AWS evidence only after the platform lifecycle shows the corresponding stage.
- Finish by proving both the public application and cleanup lifecycle.

Current limitations and next engineering steps

The implementation is intentionally bounded; future work expands scale without weakening determinism.

Current limitation	Next step
JavaScript/Python-focused deployment contract	add additional ecosystem contracts behind explicit admission
AWS-primary runtime	provider adapters and capability contracts for other compute backends
Per-project resource overhead	shared ingress, resource pooling and bin-packing
Basic release promotion	canary / weighted traffic / blue-green policies
Security scanning focused on image vulnerabilities	SBOM, provenance signatures and stronger supply-chain policy
AI advisory troubleshooting	richer SLO/evidence correlation while preserving deterministic authority

What we implemented

DeployGuard is a control plane, not a collection of cloud scripts.

Core contribution

- Immutable source/build/runtime identity
- Deterministic admission and configuration
- Governed cloud lifecycle
- Verified stable-release promotion
- Release generations + rollback/destroy
- Structured failure ownership and evidence

Integrated product capabilities

- Managed databases + persistence
- Security and federated cloud identity
- Observability and state reconciliation
- FinOps + billing
- AI-assisted troubleshooting
- Developer and operator interfaces

The FYP-II value is the implemented lifecycle correctness that connects these capabilities into one product.



DeployGuard

Repository → deterministic contracts → immutable image → AWS runtime → verified stable release

FYP-II Implementation • Questions & Live Demo

Hasan Sajjad • Faria Fatima • Tania Khawar